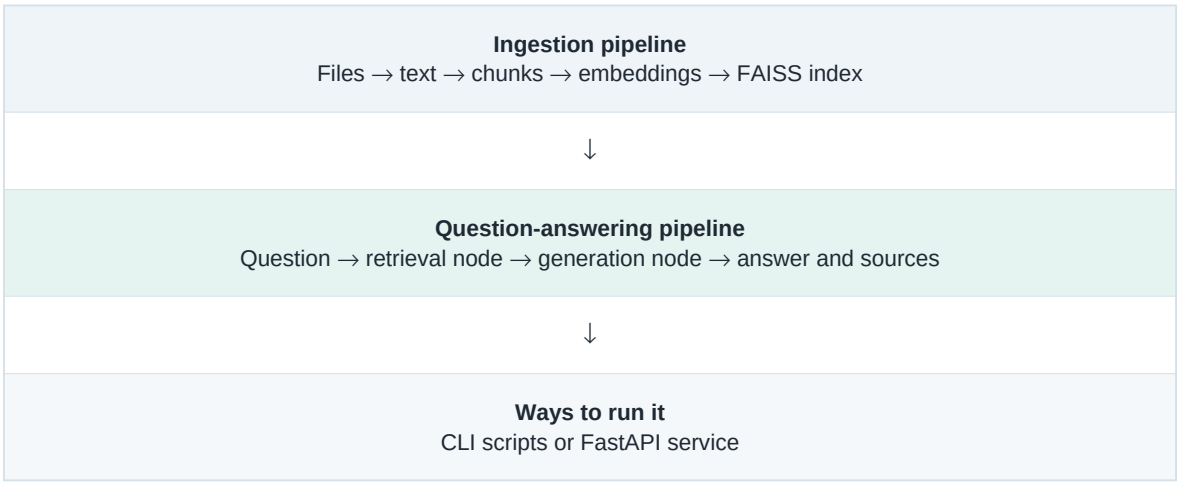
ENTERPRISE RAG WITH LANGGRAPH

Detailed codebase documentation

A practical guide to the modular Python application: document ingestion, chunking, embeddings, FAISS retrieval, LangGraph orchestration, FastAPI endpoints, configuration, testing, and production hardening.



Version 1.0 · Prepared from the current enterprise-rag-langgraph-boilerplate source tree

Scope note: the supplied boilerplate currently loads local PDF, TXT, and Markdown files. Azure Blob Storage was discussed as a future loader integration but is not part of the current ZIP implementation.

Contents

Section	What it covers
1	Purpose and architecture
2	Repository layout and module responsibilities
3	Configuration and environment setup
4	Flow A — ingestion: documents to vector index
5	Flow B — answering: question to grounded response
6	LangGraph state and graph orchestration
7	FastAPI application and API contract
8	CLI workflows and commands
9	Data structures, metadata, and source references
10	Testing and troubleshooting
11	Security, reliability, and production gaps
12	Recommended reading order and extension roadmap
Appendix	Key file map and glossary

How to use this document
Keep the PDF open while viewing the repository in VS Code. Follow the execution order rather than reading files alphabetically. For every function, identify its input, output, and the next function that consumes that output.

1. Purpose and architecture

This project is a modular Retrieval-Augmented Generation (RAG) service. It retrieves relevant passages from a local document collection and supplies those passages to a chat model so the model can answer questions grounded in the indexed material. LangGraph controls the sequence of RAG steps; FastAPI exposes the workflow as an HTTP service; command-line scripts provide a simple local way to ingest and ask questions.

The design deliberately separates two lifecycles:

- **Ingestion lifecycle:** run when the knowledge collection needs to be built or rebuilt. It extracts document text, chunks it, embeds the chunks, and saves a FAISS index.
- **Question lifecycle:** run once per question. It embeds the question, searches the existing index, supplies retrieved context to the chat model, and returns an answer with source references.

High-level architecture

INGESTION	QUESTION ANSWERING
scripts/ingest.py	scripts/ask.py or HTTP POST /api/v1/ask
ingestion/pipeline.py	workflows/graph.py
loaders.py → chunking.py	rag/graph.py
models/embeddings.py	retrieve_node → generate_node
retrieval/store.py builds index	retrieval/store.py searches index
data/vectorstore/	Answer + source references

Key architectural principle
The ingestion flow creates the searchable knowledge base. The query flow reuses it. Do not re-read and re-embed every PDF for every user question.

Technology stack

Technology	Role in this project
Python 3.11+	Application language.
uv	Dependency and project environment management.
LangChain	Document abstractions, text splitters, embedding/chat model adapters, prompts.
LangGraph	Defines executable graphs and the order of node execution.
OpenAI embeddings	Converts chunks and questions into vectors.
OpenAI chat model	Generates the answer from the question and retrieved context.
FAISS	Local vector similarity index persisted on disk.
FastAPI + Uvicorn	HTTP API and development server.
Pydantic Settings	Reads and validates application configuration.
structlog	Structured JSON logging.

2. Repository layout and responsibilities

The code is organized by responsibility. This keeps API concerns separate from retrieval logic, model setup, and document ingestion.

Path	Responsibility / how it participates
scripts/ingest.py	Executable entry point for building the vector index.
scripts/ask.py	Interactive terminal loop for asking questions.
src/enterprise_rag/main.py	Creates the FastAPI app, configures logging, registers routes, and exposes /health.
config/settings.py	Loads environment settings, defaults, and chunking validation.
ingestion/loaders.py	Scans the local document directory and loads supported file formats.
ingestion/chunking.py	Splits LangChain Document objects into overlapping chunks.
ingestion/pipeline.py	Coordinates the ingestion stages.
models/embeddings.py	Creates the OpenAI embedding client.
models/chat.py	Creates the OpenAI chat model client.
retrieval/store.py	Builds, persists, loads, and searches the FAISS index.
rag/state.py	Types the state exchanged between RAG graph nodes.
rag/prompts.py	Defines the system and human prompt template.
rag/nodes.py	Implements retrieval and answer-generation nodes.
rag/graph.py	Builds the reusable retrieve → generate LangGraph subgraph.
workflows/state.py	Types the higher-level workflow state.
workflows/graph.py	Wraps the RAG graph in an application-level workflow.
workflows/nodes.py	Reserved extension point; currently contains no executable nodes.
api/schemas.py	Validates request and response data structures.
api/routes.py	Implements POST /ask, validation, workflow invocation, and HTTP errors.
security/authorization.py	Explicit placeholder; real document access policy is not implemented.
observability/logging.py	Configures structured logs and returns module loggers.
configs/config.yaml	Documents chunking/retrieval values, but current Settings loads values from environment/defaults, not this YAML.
tests/	Unit tests for chunking and schemas, plus an API health integration test.
Dockerfile / docker-compose.yml	Container image and local container orchestration setup.

Do not confuse similarly named files
src/enterprise_rag/main.py starts the HTTP application. **scripts/ingest.py** builds the index. **scripts/ask.py** asks questions from the terminal. These are different entry points for different jobs.

3. Configuration and environment setup

The settings class in src/enterprise_rag/config/settings.py uses Pydantic Settings. It reads values from a .env file and uses defaults for settings not supplied. get_settings() is cached with lru_cache, so the application typically creates the Settings object once per process.

Important settings

Environment variable	Default	Meaning
OPENAI_API_KEY	Required	API key used by the embedding and chat clients. Keep it out of source control.
OPENAI_CHAT_MODEL	gpt-4o-mini	Chat model used for final answer generation.
OPENAI_EMBEDDING_MODEL	text-embedding-3-small	Model used to embed both indexed chunks and user queries.
APP_ENV	local	Environment label returned by /health.
LOG_LEVEL	INFO	Logging level.
API_PREFIX	/api/v1	Prefix added to the RAG API route.
DOCUMENTS_DIR	data/documents	Local input directory scanned during ingestion.
VECTORSTORE_DIR	data/vectorstore	Directory where FAISS files are saved and loaded.
CHUNK_SIZE	1000	Approximate maximum chunk size in characters.
CHUNK_OVERLAP	150	Repeated characters between adjacent chunks.
RETRIEVAL_K	5	Default maximum number of chunks returned by similarity search.
MAX_QUESTION_LENGTH	10000	Maximum question length accepted by the API route.

Setup commands

```
uv sync
cp .env.example .env
# Edit .env and set OPENAI_API_KEY
mkdir -p data/documents
# Copy PDF, TXT, or Markdown files into data/documents/
uv run python scripts/ingest.py
```

Configuration validation

Settings validate that chunk_size is positive, chunk_overlap is non-negative, and overlap is smaller than chunk_size. The chunk splitter repeats its own validation before splitting. If overlap is equal to or greater than chunk size, ingestion raises a ValueError rather than creating invalid chunks.

> **Configuration caveat**
> configs/config.yaml contains chunk_size, chunk_overlap, and retrieval_k values, but the current code does not load this YAML file. The active values come from .env and defaults in Settings. Editing config.yaml alone will not change runtime behavior.

4. Flow A — ingestion: documents to vector index

Purpose: turn source documents into a persistent index that can be searched semantically. In the current implementation, ingestion is a batch operation started by a script.

Execution sequence

Order	File / function	What happens
1	scripts/ingest.py	Imports and calls ingest_documents() when executed as the main script.
2	ingestion/pipeline.py	Gets settings and calls the loader, splitter, embedding factory, and index builder.
3	ingestion/loaders.py	Walks the configured directory recursively, loads supported files, and adds source metadata.
4	ingestion/chunking.py	Splits page/document text into smaller chunks while carrying metadata forward.
5	models/embeddings.py	Returns an OpenAIEmbeddings client configured with the selected model and key.
6	retrieval/store.py	Embeds the chunks, creates the FAISS index, and saves it under the configured directory.
7	pipeline.py	Logs completion counts and returns the number of chunks created.

4.1 Entry point: scripts/ingest.py

```
from enterprise_rag.ingestion.pipeline import ingest_documents

if __name__ == "__main__":
    print(f"Successfully indexed {ingest_documents()} chunks.")
```

The if __name__ == '__main__' guard means the call runs when this file is executed directly. Importing this file from another module will not automatically start ingestion.

4.2 Pipeline coordinator: ingestion/pipeline.py

```
def ingest_documents() -> int:
    settings = get_settings()
    documents = load_documents(settings.documents_dir)
    chunks = split_documents(
        documents, settings.chunk_size, settings.chunk_overlap
    )
    build_and_save_index(
        chunks, get_embeddings(), settings.vectorstore_dir
    )
    logger.info(
        "ingestion_completed",
        documents=len(documents),
        chunks=len(chunks),
    )
    return len(chunks)
```

Read this function from top to bottom. Each line calls a specialized module and passes its output to the next stage. The pipeline itself does not parse PDFs, calculate vector math, or implement FAISS search; it orchestrates those responsibilities.

4.3 Loading source files: ingestion/loaders.py

- Checks that the input directory exists; otherwise raises FileNotFoundError.
- Walks the directory recursively in sorted path order.
- Uses PyPDFLoader for .pdf files.
- Uses TextLoader with UTF-8 for .txt, .md, and .markdown files.
- Skips unsupported file types.
- Adds source and file_name metadata to every loaded Document.

- Raises ValueError if no supported documents were found.

PDF loader output commonly contains one Document per page, with page metadata supplied by the loader. Text files may be loaded as one Document per file. The exact number of Document objects therefore depends on file type and loader behavior.

4.4 Chunking: ingestion/chunking.py

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=chunk_size,
    chunk_overlap=chunk_overlap,
    add_start_index=True,
)
return splitter.split_documents(documents)
```

RecursiveCharacterTextSplitter tries to split at natural separators before falling back to smaller boundaries. add_start_index=True adds a start_index metadata field where supported. Metadata such as source and page is propagated to the resulting chunks.

4.5 Embeddings and index persistence

get_embeddings() returns OpenAIEmbeddings configured with OPENAI_EMBEDDING_MODEL and OPENAI_API_KEY. build_and_save_index() rejects an empty chunk list, creates the target directory, calls FAISS.from_documents(chunks, embeddings), and saves the index with save_local(). This is the point where the chunk text is embedded and associated with a searchable FAISS index.

Ingestion behavior to know
This starter rebuilds and overwrites the local FAISS index from the current local document collection each time ingestion runs. It does not yet implement incremental updates, content-hash checks, deletion synchronization, queues, or an ingestion manifest.

5. Flow B — answering: question to grounded response

Purpose: accept a user question, retrieve relevant chunks from the saved index, and use those chunks as context for a chat model. This flow should not re-ingest all documents.

Execution sequence

Order	File / function	What happens
1	scripts/ask.py or api/routes.py	Receives the question from the terminal or HTTP request.
2	workflows/graph.py	Calls the compiled high-level workflow.
3	rag/graph.py	Runs retrieve node, then generate node.
4	rag/nodes.py: retrieve_node	Passes the question to retrieve().
5	retrieval/store.py: retrieve	Loads FAISS and runs similarity search for the query.
6	rag/nodes.py: generate_node	Builds context and source list, then invokes prompt + chat model.
7	workflows/graph.py	Returns the final state to the original caller.
8	CLI or API	Prints or serializes the answer and source references.

5.1 Terminal entry point: scripts/ask.py

```
result = workflow.invoke({"question": question})
print(f"\nAnswer: {result.get('answer', '')}")
for source in result.get("sources", []):
    print(f"- {source['source']} (page {source['page']})")
```

The script runs an interactive loop. It waits for input, ignores blank questions, exits on 'exit' or 'quit', and invokes the same workflow for each non-empty question.

5.2 Retrieval: retrieval/store.py

```
def retrieve(query: str, k: int | None = None) -> list[Document]:
    if not query.strip():
        return []
    settings = get_settings()
    return load_index(
        get_embeddings(), settings.vectorstore_dir
    ).similarity_search(query, k=k or settings.retrieval_k)
```

The query is embedded with the same configured embedding model used during ingestion. FAISS compares that vector with the vectors stored in the index and returns the top k similar Documents. Similarity is a semantic matching signal, not a guarantee that a chunk fully answers the question.

5.3 Generation: rag/nodes.py

- Reads question and retrieved Documents from RAGState.
- Builds a context string. Each chunk is prefixed with source and page metadata.
- Builds a separate source-reference list for the final response.
- If context is empty, returns a fallback message and no sources.
- Otherwise, pipes ANSWER_PROMPT into get_chat_model() and invokes it with question and context.
- Returns answer and sources as a state update.

5.4 Prompt and chat model

rag/prompts.py defines a system message instructing the model to answer only from reference context, treat retrieved context as untrusted data, ignore embedded instructions that attempt to override system rules, and say when the documents do not support an answer. The human message contains the user question. models/chat.py creates ChatOpenAI with the configured chat model, temperature 0, timeout of 30 seconds, and up to two retries.

Grounding is a design goal, not a formal guarantee
The prompt encourages grounded answers, but retrieval quality, model behavior, and the quality of source documents still matter. A production system should evaluate answer faithfulness, citation correctness, retrieval recall, and refusal behavior.

6. LangGraph state and orchestration

LangGraph describes a workflow as a graph of nodes and edges. A node is a function or subgraph that does work. An edge determines which node runs next. State is the shared structured data that nodes read and update.

6.1 The inner RAG graph

```
graph = StateGraph(RAGState)
graph.add_node("retrieve", retrieve_node)
graph.add_node("generate", generate_node)
graph.add_edge(START, "retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", END)
rag_graph = graph.compile()
```

This graph is sequential: START → retrieve → generate → END. There is no conditional routing in the current implementation. The graph is compiled at module import time and reused by callers in that Python process.

6.2 The outer workflow graph

```
builder = StateGraph(WorkflowState)
builder.add_node("rag", rag_graph)
builder.add_edge(START, "rag")
builder.add_edge("rag", END)
workflow = builder.compile()
```

The outer workflow wraps the RAG subgraph as one node. Today, that is a thin wrapper. It establishes a place to add future steps such as intent classification, retrieval strategy selection, tool calling, answer validation, or human approval without mixing those responsibilities into the retrieval/generation code.

6.3 State fields

Field	Type / purpose
question	str — user's original question.
request_id	str — request correlation identifier; normally generated by the API route.
documents	list[Document] — retrieved chunks and their metadata.
answer	str — generated response.
sources	list[dict[str, str]] — source and page values returned with the answer.
error	str — reserved state field for error propagation; not actively populated by current graph nodes.

RAGState and WorkflowState are TypedDict definitions with total=False. This means keys are optional at the type level. Each node returns only the fields it updates; LangGraph merges these updates into the running state. For example, retrieve_node returns documents, then generate_node returns answer and sources.

6.4 Why keep both graphs?

- **RAG subgraph:** reusable retrieval-and-generation capability.
- **Workflow graph:** application-level orchestration where business routing and other stages can be added.
- **API:** transport layer; it should not contain the retrieval algorithm or prompt construction.

This separation keeps the application maintainable as the number of workflow steps grows. It is not yet a multi-agent system: there are currently two deterministic RAG nodes and one outer wrapper node.

7. FastAPI application and API contract

The HTTP application is defined in src/enterprise_rag/main.py. Run it with Uvicorn; do not confuse this file with the ingestion script.

```
uv run uvicorn enterprise_rag.main:app --reload
```

7.1 What main.py does

- Calls configure_logging() to set up structured logging.
- Calls get_settings() to load application configuration.
- Creates a FastAPI object titled Enterprise RAG API.
- Registers the router from api/routes.py under API_PREFIX, default /api/v1.
- Exposes GET /health returning status and environment.

7.2 Ask endpoint

The route is registered at POST /api/v1/ask. It receives an AskRequest, trims the question, checks that it is not blank and does not exceed MAX_QUESTION_LENGTH, creates a UUID request ID, invokes the workflow, and returns an AskResponse.

Example request

```
POST /api/v1/ask
Content-Type: application/json

{
  "question": "What are the common causes of battery failure?"
}
```

Example response shape

```
{
  "request_id": "generated-uuid",
  "answer": "Answer grounded in the retrieved document context...",
  "sources": [
    {"source": "data/documents/guide.pdf", "page": "4"}
  ]
}
```

The answer text and source values above are illustrative, not a guaranteed result. Source page values are converted to strings by the response schema. Actual metadata depends on the loader and documents indexed.

7.3 Error behavior

Condition	Current behavior
Blank or invalid request body	Pydantic validation or explicit route validation returns a 422 response.
Question too long in route	Returns HTTP 413.
FAISS index missing	Logs an error and returns HTTP 503 with 'Knowledge index is not ready.'
Unexpected error	Logs an exception and returns HTTP 500 with a reference ID.
Health check	Returns HTTP 200 if the application can respond; does not test LLM or vector index readiness.

7.4 Run and test the API

```
# Ingest documents first
uv run python scripts/ingest.py

# Start the API
uv run uvicorn enterprise_rag.main:app --reload

# Browse interactive OpenAPI docs
http://127.0.0.1:8000/docs

# Health check
http://127.0.0.1:8000/health
```

> **Important distinction**
> The API and CLI share the same compiled workflow. The CLI prints the result; FastAPI validates HTTP input and serializes the result as JSON.

8. Operational workflows and commands

Use the commands below from the project root, where pyproject.toml is located.

Goal	Command	Notes
Install dependencies	uv sync	Creates/synchronizes the uv environment from pyproject.toml and lock data when available.
Configure environment	cp .env.example .env	Edit .env and set OPENAI_API_KEY.
Index documents	uv run python scripts/ingest.py	Reads DOCUMENTS_DIR and rebuilds the FAISS index.
Ask from terminal	uv run python scripts/ask.py	Interactive question loop.
Start API	uv run uvicorn enterprise_rag.main:app --reload	Development server; reloads on code changes.
Run tests	uv run pytest	Runs tests configured under tests/.
Build container	docker compose up --build	Uses Dockerfile and docker-compose.yml; .env must exist.

Typical local development session

```
# 1. Install and configure
uv sync
cp .env.example .env
# Put documents in data/documents/

# 2. Build or rebuild the index
uv run python scripts/ingest.py

# 3A. Ask in terminal
uv run python scripts/ask.py

# 3B. Or start the HTTP service in another terminal
uv run uvicorn enterprise_rag.main:app --reload
```

The vector index is a generated artifact. If you change the source documents, chunking settings, or embedding model, rebuild the index. If you change the embedding model, the stored vectors may no longer be compatible with the model used for queries; re-indexing is the safe approach.

Docker notes

The Dockerfile copies pyproject.toml, README.md, src, and scripts, synchronizes runtime dependencies, and starts Uvicorn. docker-compose.yml maps port 8000 and mounts the local data directory at /app/data. The .env file is passed into the container. For production, do not assume this local volume or a local FAISS index is shared safely across multiple replicas.

9. Data structures, metadata, and source references

LangChain Document is the primary object passed through ingestion and retrieval. It keeps text and metadata together so that the system can cite where a retrieved passage came from.

Property	Meaning in this project
page_content	Text content extracted from a source and/or contained in a chunk.
metadata['source']	Source file path set by loaders.py.
metadata['file_name']	Basename of the source file.
metadata['page']	Page number supplied by the PDF loader when available; the exact indexing convention is loader-defined.
metadata['start_index']	Character start position added by the chunk splitter when available.

When retrieval returns Documents, generate_node builds two outputs from them:

- **Context string:** the retrieved chunk text with a visible source and page label, passed to the LLM.
- **Sources array:** structured source/page pairs returned to the API or CLI.

The current source list is based on retrieved chunks, so multiple chunks from the same source may result in repeated source entries. It is not yet a citation-verification layer: it does not prove that every factual sentence in the generated answer is supported by a specific source.

Metadata becomes especially important in an enterprise system. Common future fields include document_id, tenant_id, allowed_groups, content_hash, document_version, ingestion_timestamp, and source_uri. Those fields enable document lifecycle tracking, filtering, authorization, and citation links.

Access-control warning
The current retrieval function performs similarity search without user identity or document-level filters. security/authorization.py is only a placeholder that raises AuthorizationNotConfiguredError. Do not expose confidential documents to users until authorization is implemented and enforced during retrieval.

10. Testing and troubleshooting

The repository includes starter tests. They verify basic behavior, not full production correctness.

Test	What it checks
tests/unit/test_chunking.py::test_chunking_retains_metadata	Chunking creates multiple chunks and carries source metadata.
tests/unit/test_chunking.py::test_invalid_overlap_raises	Invalid overlap values raise ValueError.
tests/unit/test_schemas.py::test_empty_question_rejected	AskRequest rejects an empty question.
tests/unit/test_schemas.py::test_valid_question	A valid question can be parsed by the request schema.
tests/integration/test_api.py::test_health_endpoint	FastAPI /health endpoint responds with status 200 and status=ok.

Run the tests

```
uv run pytest
```

Common failures

Symptom	Likely cause	What to check
OPENAI_API_KEY validation error	.env is missing or key is empty.	Ensure .env exists at project root and OPENAI_API_KEY is populated.
Documents directory missing	DOCUMENTS_DIR points to a missing folder.	Create data/documents or update DOCUMENTS_DIR.
No supported documents found	Folder contains no PDF/TXT/Markdown files.	Check extensions and path; unsupported formats are skipped.
Cannot index empty chunks	No usable text was extracted or splitting yielded no chunks.	Open source documents; scanned PDFs may require OCR.
Vector index missing	Ingestion has not run successfully or directory is wrong.	Run scripts/ingest.py and check VECTORSTORE_DIR.
Bad embedding/model request	Wrong model name, credentials, network access, or provider issue.	Check API key, model configuration, quotas, connectivity, and logs.
Answers are irrelevant	Poor extraction, chunking, retrieval, or question formulation.	Inspect retrieved chunks; tune chunking and retrieval; add evaluation.
Repeated or stale content	Current pipeline rebuilds index but no incremental lifecycle exists.	Re-ingest after source changes and confirm the expected index directory.
API returns 503	Index not ready.	Build index before querying.

Debugging technique

Trace one question manually: identify the initial question state; inspect what retrieve_node passes to retrieve(); inspect which Documents FAISS returns; verify their source/page metadata; then inspect the context string sent to the model. This isolates whether the issue is ingestion, retrieval, prompting, or generation.

11. Security, reliability, and production gaps

This repository is a starter scaffold, not a production-certified service. The README correctly calls out several pieces that must be completed before handling sensitive or business-critical data.

Area	Current state	Production work
Authentication	No user authentication in the API route.	Integrate enterprise identity provider and validate tokens.
Document authorization	Placeholder only; not invoked by retrieval.	Apply tenant and document ACL filters inside retrieval, before context is sent to the model.
Secrets	API key is read from .env.	Use managed secrets; avoid logging credentials; rotate keys.
Vector storage	Local FAISS index.	Choose managed vector database/search service for shared, scalable deployment.
Index persistence	FAISS load uses allow_dangerous_deserialization=True.	Only load trusted index files; protect storage and deployment artifacts.
Incremental ingestion	Whole index rebuilt from current folder.	Track hashes/ETags and versions; upsert changed chunks and delete stale chunks.
PDF OCR	PyPDF text extraction.	Add OCR/layout parsing for scanned or complex PDFs.
Evaluation	No RAG quality evaluation suite.	Measure retrieval recall, faithfulness, relevance, latency, and cost on a test set.
Observability	Structured logs; basic events.	Add distributed traces, metrics, dashboards, alerts, and redaction.
Rate limiting	Not implemented.	Add API quotas, request limits, concurrency controls, and abuse protection.
Prompt injection	Prompt tells model to treat context as untrusted.	Add defense-in-depth, document sanitization, tool restrictions, and adversarial tests.
Graph persistence	No checkpointer configured.	Add durable checkpointing only if workflows need resumability or conversation state.
Reliability	Limited retry behavior at chat model layer.	Add provider-aware retry/backoff, timeouts, idempotency, and dead-letter handling for ingestion.

Azure Blob Storage integration

The current ZIP reads local files. For Azure Blob Storage, replace or extend the loading stage: authenticate using an approved credential (for example, managed identity in Azure), list eligible PDF blobs, download each blob or stream it to a temporary file, extract page text, and add stable metadata such as blob name, blob URI, ETag/content hash, and document ID. The existing chunking and embedding stages can then consume the extracted Document objects.

For production, do not download every PDF into one giant in-memory list. Process documents in bounded batches, log per-document outcomes, retry transient failures, and maintain a manifest so unchanged documents can be skipped. A managed vector search service is usually a better fit than local FAISS for multiple API replicas and access-control filtering.

Do not treat a prompt as access control
The model prompt is not a security boundary. Confidentiality must be enforced in application code before retrieved text is passed to the LLM.

12. Recommended reading order and extension roadmap

For learning, follow the same order as runtime execution. Read each function and write down what it accepts and what it returns.

Step	Open this file	Question to answer while reading
1	scripts/ingest.py	What starts ingestion?
2	ingestion/pipeline.py	What function is called next, and what does it return?
3	ingestion/loaders.py	How are files converted into Documents?
4	ingestion/chunking.py	How are Documents split and metadata preserved?
5	models/embeddings.py	Which model converts text to vectors?
6	retrieval/store.py	How is the index built, saved, loaded, and searched?
7	scripts/ask.py	How does the terminal submit a question?
8	workflows/graph.py	What is the outer graph responsible for?
9	workflows/state.py + rag/state.py	Which fields travel through the graph?
10	rag/graph.py	Which node runs first and what is the edge order?
11	rag/nodes.py	What does retrieval return, and how is the prompt built?
12	rag/prompts.py + models/chat.py	What rules are sent to the LLM and which model is used?
13	api/routes.py + api/schemas.py	How is HTTP input validated and the result returned?
14	main.py	How is the FastAPI application assembled and served?

Suggested extension sequence

- **Stage 1 — make the baseline reliable:** run tests, ingest a small known PDF, inspect chunk metadata, and test several questions.
- **Stage 2 — object storage:** implement Azure Blob loader and test permissions, download failures, PDF parsing, and stable metadata.
- **Stage 3 — document lifecycle:** add hash/ETag tracking, idempotent ingestion, updates/deletes, and a processing manifest.
- **Stage 4 — retrieval quality:** add metadata filters, optional reranking, relevance thresholds, and a retrieval evaluation dataset.
- **Stage 5 — enterprise security:** implement identity, ACL enforcement, tenant isolation, secret management, and audit logging.
- **Stage 6 — operations:** add traces/metrics, alerts, rate limits, load tests, deployment configuration, and a managed vector index if multiple replicas are required.

Appendix A. Key file map

Question	Start here
Where do I put local documents?	data/documents/
Where do I configure the OpenAI key?	.env (copy from .env.example).
Where is the ingestion entry point?	scripts/ingest.py
Where is document loading implemented?	src/enterprise_rag/ingestion/loaders.py
Where is chunking implemented?	src/enterprise_rag/ingestion/chunking.py
Where is the vector index built and searched?	src/enterprise_rag/retrieval/store.py
Where is the chat model created?	src/enterprise_rag/models/chat.py
Where is the embedding model created?	src/enterprise_rag/models/embeddings.py
Where is RAG graph node order defined?	src/enterprise_rag/rag/graph.py
Where are retrieval and generation nodes?	src/enterprise_rag/rag/nodes.py
Where is the outer workflow assembled?	src/enterprise_rag/workflows/graph.py
Which file starts the API?	src/enterprise_rag/main.py via Uvicorn.
Where is the question endpoint?	src/enterprise_rag/api/routes.py
Where are request and response models?	src/enterprise_rag/api/schemas.py

Appendix B. Glossary

Term	Plain-language explanation
RAG	Retrieval-Augmented Generation: retrieve relevant source text, then ask a language model to answer using that context.
Document	A text unit plus metadata; in LangChain represented by Document.
Chunk	A smaller piece of a document used for embedding and retrieval.
Embedding	A numerical vector representing semantic characteristics of text.
Vector store / index	A structure that stores vectors and finds vectors similar to a query vector.
FAISS	A library for efficient similarity search over dense vectors; here persisted locally.
Retriever	The component that selects relevant chunks for a question.
Prompt	Instructions and input messages sent to the chat model.
LangGraph node	A function or subgraph that performs one step in a graph.
LangGraph edge	A transition that defines which node runs next.
State	Structured data passed between graph nodes.
FastAPI route	An HTTP endpoint that accepts a request and returns a response.
Metadata filter	A condition such as tenant ID or document permissions used to constrain retrieval.
Reranking	A second-stage model or algorithm that reorders retrieved chunks by relevance.

Final mental model
Ingestion: source files → extracted text → chunks → vectors → saved index.
Answering: question → query vector → relevant chunks → prompt + chat model → answer and sources.
Orchestration: LangGraph defines the node sequence; FastAPI or the CLI invokes that workflow.

Document generated from the repository structure and source files available in the current workspace. Where a capability is only a placeholder or was discussed as a future integration, it is explicitly described as not yet implemented.